

Exercise 1: Neural Networks: Basic Components

Lecturer: Aurelien Lucchi

Teaching Assistants: Nadezhda Ilieva, Jim Zhao

Problem 1 (Simple calculations on multi-layer perceptrons):

Consider a L -layer multi-layer perceptron (MLP) without bias, with input $\mathbf{x} \in \mathbb{R}^d$, outputs $\mathbf{y} \in \mathbb{R}^k$, with a hidden layer of width w for $L \geq 2$. This MLP consists thus of layers $\mathbf{W}_1 \in \mathbb{R}^{w \times d}, \dots, \mathbf{W}_L \in \mathbb{R}^{k \times w}$, and $\mathbf{W}_i \in \mathbb{R}^{w \times w}$ for $1 < i < L$.

- What is the total number of parameters of the MLP and how does it scale with the depth and the width of the model?
- How many scalar addition and multiplication operations are needed for a single forward pass?
Hint: Consider first what the complexity of a single matrix-matrix or matrix-vector multiplication of some given dimensions is.
- How does the computational complexity of a single forward pass scale with depth L and width w ?
- (Bonus) What is the computational complexity of a single backward pass via backpropagation?

Problem 2 (Sigmoid vs. Hyperbolic Tangent):

For any activation function $\sigma : \mathbb{R} \rightarrow \mathbb{R}$, we extend its definition to multivariate case by entry-wise evaluation:

$$\sigma : \mathbb{R}^n \rightarrow \mathbb{R}^n, (x_i)_{i=1}^n \mapsto (\sigma(x_i))_{i=1}^n.$$

Consider the following two-layer feedforward network with $\left(\tanh : \mathbb{R} \rightarrow \mathbb{R}, x \mapsto \frac{e^x - e^{-x}}{e^x + e^{-x}}\right)$ as the activation function:

$$F : \mathbb{R}^d \rightarrow \mathbb{R}^m, \mathbf{x} \mapsto \mathbf{W}_2 \tanh(\mathbf{W}_1 \mathbf{x} + b_1) + b_2,$$

where $\mathbf{x} \in \mathbb{R}^d$, $\mathbf{W}_1 \in \mathbb{R}^{n \times d}$, $b_1 \in \mathbb{R}^n$, $\mathbf{W}_2 \in \mathbb{R}^{m \times n}$, $b_2 \in \mathbb{R}^m$.

- Show that there exists a two-layer network

$$G(\mathbf{x}) = \mathbf{W}'_2 s_1(\mathbf{W}'_1 \mathbf{x} + b'_1) + b'_2,$$

which computes exactly the same function as $F(x)$ using weight matrices and bias vectors of the same dimension, but with the sigmoid activation function s_1 . Write down the weights $\mathbf{W}'_1, \mathbf{W}'_2, b'_1, b'_2$ in terms of $\mathbf{W}_1, \mathbf{W}_2, b_1, b_2$.
(Hint: You can first show that $\tanh(x) = 2s_1(2x) - 1$, where s_1 is the Sigmoid function.)

- Does the same statement hold if s_1 is replaced by l_1 ? Why?

Problem 3 (Gradient for Three Layer network with softmax activation and cross-entropy loss):

Consider a three layer neural network with a softmax activation at the output defined as

$$f_{\mathbf{W}, \Theta}(\mathbf{x}) = \sigma^{\max}(\mathbf{W}\mathbf{x}; \Theta),$$

where the softmax function converts the logits $\mathbf{W}\mathbf{x}$ into a probability distribution via

$$\sigma_i^{\max}(\mathbf{x}; \Theta) = \frac{\exp(\mathbf{x}^\top \theta_i)}{\sum_{j=1}^k \exp(\mathbf{x}^\top \theta_j)}, \quad \Theta = [\theta_1, \dots, \theta_k].$$

The cross-entropy loss is chosen to train the network, i.e.

$$\ell(\mathbf{x}, \mathbf{y}; \mathbf{W}, \Theta) = -\mathbf{y}^\top \ln(\sigma^{\max}(\mathbf{W}\mathbf{x}; \Theta))$$

- Derive the derivative with respect to each θ_i , i.e. $\frac{\partial \ell(\mathbf{x}, \mathbf{y}; \mathbf{W}, \Theta)}{\partial \theta_i}$.
- Derive the derivative with respect to $\mathbf{W}$, i.e. $\frac{\partial \ell(\mathbf{x}, \mathbf{y}; \mathbf{W}, \Theta)}{\partial \mathbf{W}}$.

Hint: Make use of the matrix calculus to simplify the computation. Alternatively, calculate the the derivative entry-wise and combine it into a matrix expression.